Table of Contents

I. INTRODUCTION 3
II. TASK 1: THE LONG-RUNNING PROCESS 3
A. C Source Code 3
B. Terminal Execution & Verification 3
Summary 3
III. TASK 2: PROCESS IDENTITY (PID AND PPID) 4
A. C Source Code 4
Summary 4
B. Terminal Execution & Verification 4
Summary 4
IV. TASK 3: EXIT CODES AND OS FEEDBACK 5
A. C Source Code 5
B. Terminal Execution & Verification 5
Summary 5
V. TASK 4: STANDARD I/O STREAMS 6
A. C Source Code 6
B. Terminal Execution & Verification 6
Summary 6
VI. TASK 5: CONDITIONAL EXECUTION AND TERMINATION 7
A. C Source Code 7
B. Terminal Execution & Verification 7
Summary 7
VII. CONCLUSION 8

Table of Figures

Figure 1: task1 alive.c 3
Figure 2: Terminal 1: Run in background.....3
Figure 3: Terminal 2: Monitor the process 3
Figure 4: task2 identity.c.....4
Figure 5: Terminal Commands_1 4
Figure 6: Terminal Commands_2 4
Figure 11: Program with Return Value.....5
Figure 12: Checking Program Exit Status.....5
Figure 11: task4 input.c.....6
Figure 12: Running Task 4.....6
Figure 11: task5 control.c.....7
Figure 12: Testing Control Flow.....7

Lab 3: Investigating Process Lifecycles and OS Interaction

I. INTRODUCTION

In Lecture 3, we learned that an OS manages processes and resources. In this lab, we bring that theory to life. Instead of just writing code that prints text, we will write code that interacts with the Linux kernel to query its own process identity, control its execution time, and report its success or failure back to the OS shell.

II. TASK 1: THE LONG-RUNNING PROCESS

OS Insight: In enterprise servers, processes run for days or months. We must know how to monitor them while they are active.

A. C Source Code

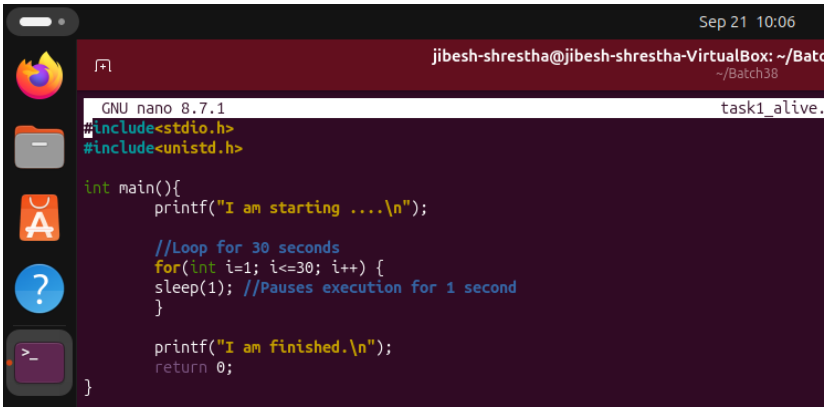


Figure 1: task1 alive.c

B. Terminal Execution & Verification

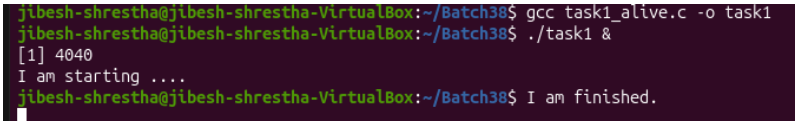


Figure 2: Terminal 1: Run in background



Figure 3: Terminal 2: Monitor the process

Summary

We create a program that runs for 30 seconds using a loop and sleep() and run the program in the background using & so we can use a second terminal to inspect it while it's alive.

III. TASK 2: PROCESS IDENTITY (PID AND PPID)

OS Insight: Every process in Linux has a unique Process ID (PID). The process that created it is the Parent Process ID (PPID), usually your terminal shell.

A. C Source Code

```c
#include<stdio.h>
#include<unistd.h>

int main() {
 // Get current Process ID
        pid_t my_pid = getpid();

        // Get Parent Process ID
        pid_t my_ppid = getppid();

        printf("My PID is: %d\n", my_pid);
        printf("My parent PID is: %d\n", my_ppid);

        printf("Sleeping for 20 seconds...\n");
        sleep(20);

        return 0;
}
```

Figure 4: task2 identity.c

Summary

We write a c code to show process ID of the currently running process.

B. Terminal Execution & Verification

```
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ gcc task2_identity.c -o task2
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ ./task2 &
[1] 4343
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ My PID is: 4343
My parent PID is: 3822
Sleeping for 20 seconds...
```

Figure 5: Terminal Commands_1

```
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ ps -p 3822 -o pid,ppid,cmd
    PID    PPID CMD
   3822    3697 /usr/bin/bash
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$
```

Figure 6: Terminal Commands_2

Summary

Verifying the PID using OS ps command.

IV. TASK 3: EXIT CODES AND OS FEEDBACK

OS Insight: When a program finishes, it returns an integer to the OS. 0 means ”Success”, and any non-zero number (like 1) means ”Error/Failure”. The shell stores this in $?.

A. C Source Code



Figure 7: Program with Return Value

B. Terminal Execution & Verification

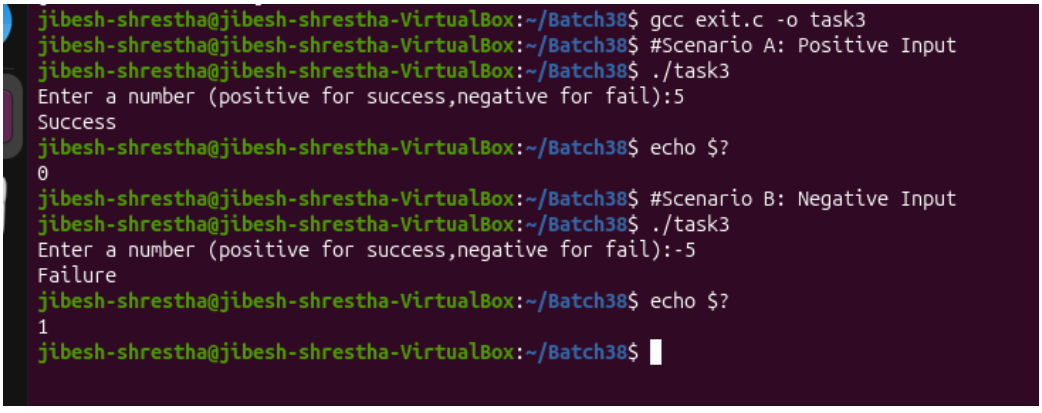


Figure 8: Checking Program Exit Status

Summary

The return value of main() indicates program success or failure to the OS

V. TASK 4: STANDARD I/O STREAMS

OS Insight: The OS provides standard input (stdin) and standard output (stdout) streams. scanf reads from stdin, and printf writes to stdout.

A. C Source Code

```c
#include <stdio.h>

int main(){
        char name[50];

        printf("Enter your name:");
        // Read string from standard input
        scanf("%s",name);
        printf("Hello, %s! Welcome to OS Class.\n",name);
        return 0;
}
```

Figure 9: task4 input.c

B. Terminal Execution & Verification

```
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ sudo nano task4_input.c
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ gcc task4_input.c -o task4
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ ./task4
Enter your name:Jibesh
Hello, Jibesh! Welcome to OS Class.
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$
```

Figure 10: Running Task 4

Summary

We wrote code using printf and scanf and understood how it works.

VI. TASK 5: CONDITIONAL EXECUTION AND TERMINATION

OS Insight: Programs often branch based on user input. The OS tracks the final exit code to determine if the script or automation pipeline should continue or halt.

A. C Source Code

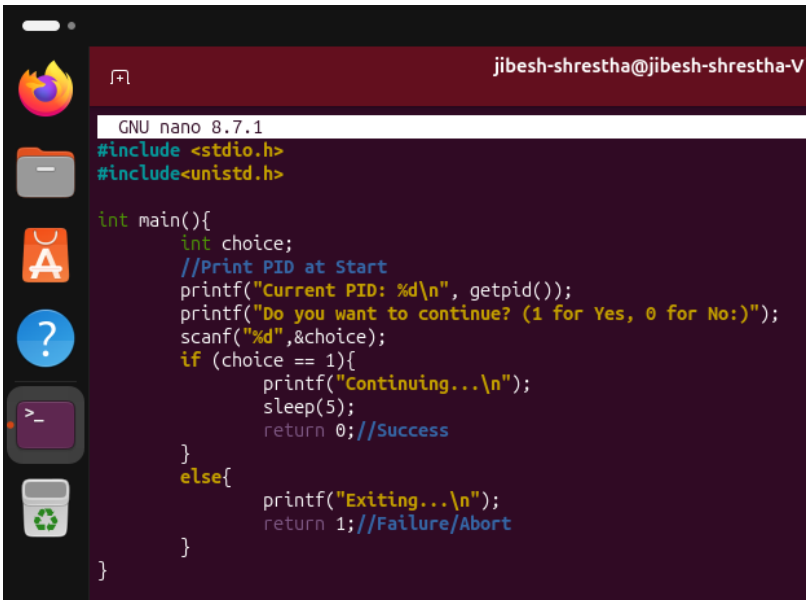


Figure 11: task5 control.c

B. Terminal Execution & Verification

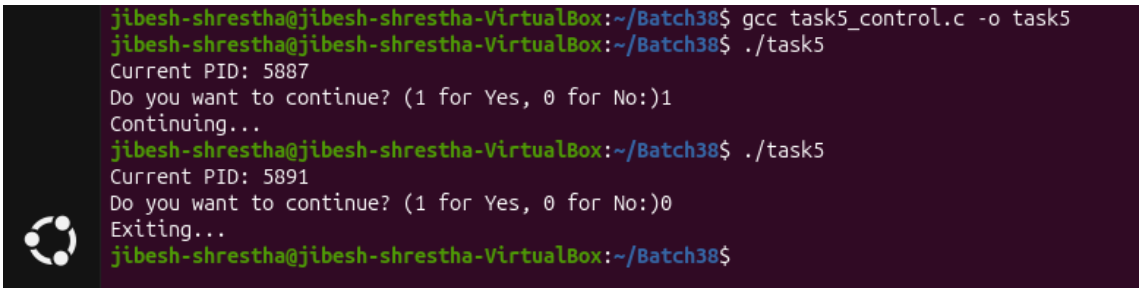


Figure 12: Testing Control Flow

Summary

We wrote code and merged the Process ID concept with user input concept.

VII. CONCLUSION

This lab demonstrated that C programs do not exist in a vacuum; they are active processes managed by the Linux kernel. By utilizing system calls like getpid(), sleep(), and return codes, we successfully bridged the gap between high-level C code and low-level OS process management.